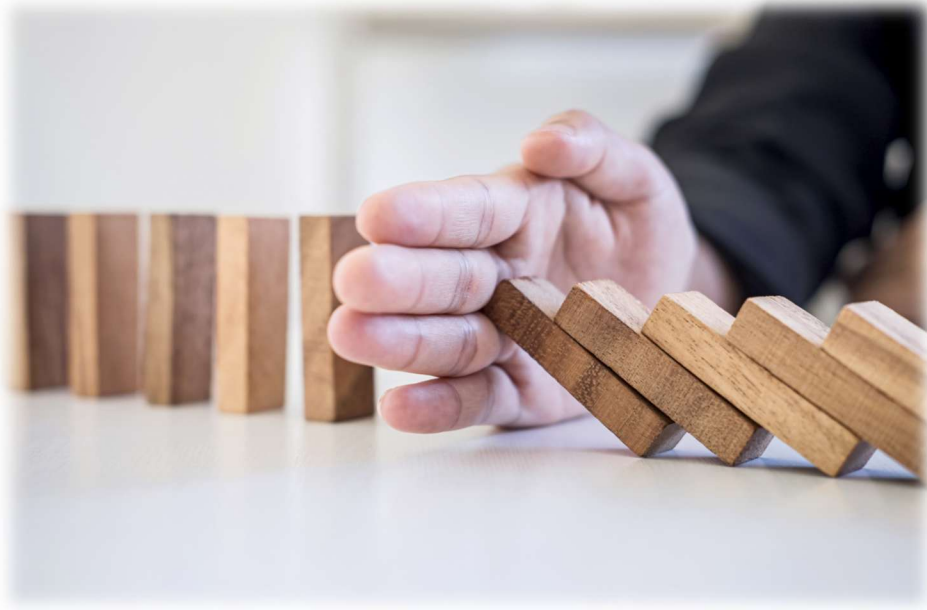


THREADS

System Calls



System Calls Project Definition
(Spring 2026)
Version 1.1

1. Introduction

The third stage of our OS development project introduces user level processes that access kernel features through system calls. We also implement functionality for semaphores based on the mailbox system developed in the Messaging project.

1.1. Objectives and Scope

The objective of this document is to specify requirements for the System Calls development project. Included is a specification for the required functions (e.g. the System Calls Application Programming Interface (API)) that must be implemented for this part of the kernel's development.

This document combined with the course's lecture and presentation materials define the totality of the System Calls project.

1.2. Target Audience

The document is intended for students taking advanced operating systems courses at the graduate or undergraduate level.

2. User Level Processes and System Calls

The System Calls project enables processes to run in *user mode* while accessing the kernel-level features developed in the Scheduler and Messaging projects. See Figure 1 for an illustration of the dependencies between the three projects.

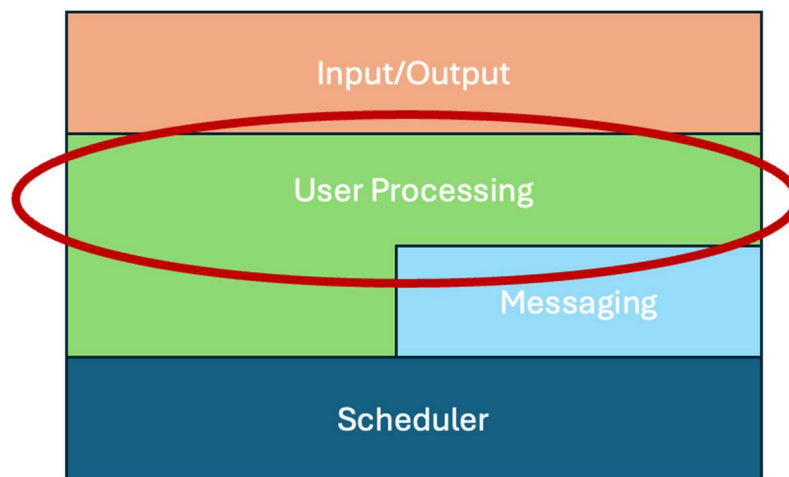


Figure 1 The System Calls project feature relies on the Messaging and Scheduler interfaces.

Processes created, or spawned, in this phase processes cannot directly access kernel-level functions like **k_spawn** or **k_wait**. Instead, the processes access these functions through a system call which traps the kernel and executes the call on the process's behalf. The kernel is trapped through an interrupt that occurs when a system call is made. This interrupt, **THREADS_SYS_CALL_INTERRUPT**, invokes a system call handler that is responsible for the system call's implementation.

Some good news is that the **THREADS_SYS_CALL_INTERRUPT** handler function is already implemented and is part of the provided Messaging project. The Messaging project sets the system call interrupt handler upon kernel start up and will invoke system calls through a system call vector table called **systemCallVector**. This vector is defined for you in the starter project. The vector holds a function pointer for each of the system calls that the kernel supports.

For the System Calls project, we are implementing the system calls defined in Table 1.

Table 1 System calls to user process mapping

System Call Id	User Process Interface
SYS_SPAWN	Spawn
SYS_WAIT	Wait
SYS_EXIT	Exit
SYS_SLEEP	SleepSeconds
SYS_SEMCREATE	SemCreate
SYS_SEMP	SemP
SYS_SEMV	SemV
SYS_SEMFREE	SemFree
SYS_GETTIMEOFDAY	GetTimeofday
SYS_CPUTIME	CPUTime
SYS_GETPID	GetPID

Your System Calls project implementation must set a handler for each of these system calls. The following code illustrates how to set a handler for the Spawn call as an example.

```
systemCallVector[SYS_SPAWN] = spawn_system_call_handler;
```

In this code snippet, the function **spawn_system_call** is your kernel-level implementation for the Spawn system call. The handler is the place where the **k_spawn** function is called. Note that the handler functions are not only responsible for performing the requested kernel operation. They must also translate parameters between the system call structure and add any functionality that is required to keep track of user-level processes.

It's important to note that this interrupt-based implementation is what separates kernel mode processes from user-mode processes: when the system call handler is called, the kernel is in kernel-mode. User-level processes use the **Spawn** interface function to invoke this process.



Important: SleepSeconds is implemented in the Devices phase, not required for this project. Processes created with **Spawn** run in user-mode and not in kernel mode.

2.1. System Call Handler Functions

Like interrupt handlers, system call handler functions have a common, structured interface based on the following prototype definition:

```
static void system_call_handler(system_call_arguments_t* args);
```

The functions do not return a value. Data is passed into the handler and out of the handler by way of the **args** parameter. The `system_call_arguments_t` data type is defined in `THREADSLib.h` and has the following definition:

```
/* structure passed to system calls. */
#define THREADS_MAX_SYSCALL_ARGUMENTS 5
typedef struct
{
    uint32_t call_id;
    intptr_t arguments[THREADS_MAX_SYSCALL_ARGUMENTS];
} system_call_arguments_t;
```

This generic structure identifies the system call with `call_id`. Each system call can have up to 5 arguments. The meaning of these arguments varies from system call to system call. See Section 4 for these details.



Important: The arguments in the `system_call_arguments_t` structure may be input parameters, output parameters or both.

3. Semaphores

The System Call project exposes several kernel level functions that were created during the Scheduler project. These include `spawn`, `wait`, `exit`, and the timing functions. We are also adding kernel functions for semaphores with the addition of the following functions:

- `k_semcreate` – Create a semaphore
- `k_semp` – Decrement the semaphore
- `k_semv` – Increment the semaphore
- `k_semfree` – Free the semaphore

Semaphore management is performed using kernel-level functions. Semaphores each have a unique identifier that is returned from the `k_semcreate` function. Semaphores are incremented and decremented using the `k_semv` and `k_semp` functions, respectively. The `k_semfree` releases a semaphore. See the detailed function definitions below.

k_semcreate

```
int k_semcreate(int initial_value)
```

Description	The k_semcreate function creates and initializes a semaphore. Semaphores are assigned a unique identifier that is used for the other semaphore functions. The initial value of the semaphore is passed as the only parameter.								
Arguments	<i>Initial_value</i>	The initial value of the semaphore.							
Return Value	<i>int</i>	The return value must be one of the following error codes: <table><tr><th>Return Value</th><th>Result</th></tr><tr><td>>= 0</td><td>The identifier of the newly created semaphore.</td></tr><tr><td>-1</td><td>The semaphore could not be created</td></tr></table>		Return Value	Result	>= 0	The identifier of the newly created semaphore.	-1	The semaphore could not be created
Return Value	Result								
>= 0	The identifier of the newly created semaphore.								
-1	The semaphore could not be created								

k_semp

```
int k_semp(int sem_id)
```

Description	The <code>k_semp</code> function decrements the value of the specified semaphore. Semaphore values cannot go below zero. The function must block a calling process when the value is zero and unblock the process once the semaphore is incremented to a value above zero. <i>There may be more than one process waiting on a semaphore to be incremented. If a semaphore is freed, all waiting processes must call <code>sys_exit(1)</code> to ensure the parent process is properly notified via <code>Wait</code>.</i>								
Arguments	<code>sem_id</code>	The semaphore to decrement.							
Return Value	<code>int</code>	The return value must be one of the following error codes: <table><tr><th>Return Value</th><th>Result</th></tr><tr><td><code>0</code></td><td>The semaphore was successfully decremented.</td></tr><tr><td><code>-1</code></td><td>An error occurred when trying to decrement the semaphore.</td></tr></table>		Return Value	Result	<code>0</code>	The semaphore was successfully decremented.	<code>-1</code>	An error occurred when trying to decrement the semaphore.
Return Value	Result								
<code>0</code>	The semaphore was successfully decremented.								
<code>-1</code>	An error occurred when trying to decrement the semaphore.								

k_semv

```
int k_semv(int sem_id)
```

Description	The <code>k_semv</code> function increments the value of the specified semaphore. This operation must take into consideration any process that is currently waiting for the semaphore to be released (or incremented). It is important to signal, or wake, the waiting process to allow that process to decrement the semaphore and continue operating.								
Arguments	<code>sem_id</code>	The semaphore to increment.							
Return Value	<code>int</code>	The return value must be one of the following error codes: <table><tr><th>Return Value</th><th>Result</th></tr><tr><td>0</td><td>The semaphore was successfully incremented.</td></tr><tr><td>-1</td><td>An error occurred when trying to decrement the semaphore.</td></tr></table>		Return Value	Result	0	The semaphore was successfully incremented.	-1	An error occurred when trying to decrement the semaphore.
Return Value	Result								
0	The semaphore was successfully incremented.								
-1	An error occurred when trying to decrement the semaphore.								

k_semfree

```
int k_semfree(int sem_id)
```

Description	The k_semfree function closes the semaphore and unblocks and notifies any waiting processes that the semaphore is being freed.										
Arguments	<i>sem_id</i>	The semaphore to free.									
Return Value	<i>int</i>	The return value must be one of the following error codes: <table><tr><th>Return Value</th><th>Result</th></tr><tr><td>0</td><td>The semaphore was successfully incremented.</td></tr><tr><td>1</td><td>The semaphore was freed, but there were processes waiting for the semaphore in semp.</td></tr><tr><td>-1</td><td>An error occurred when trying to decrement the semaphore.</td></tr></table>		Return Value	Result	0	The semaphore was successfully incremented.	1	The semaphore was freed, but there were processes waiting for the semaphore in semp.	-1	An error occurred when trying to decrement the semaphore.
Return Value	Result										
0	The semaphore was successfully incremented.										
1	The semaphore was freed, but there were processes waiting for the semaphore in semp.										
-1	An error occurred when trying to decrement the semaphore.										

4. System Call Mapping

The sections below define the interface used for each of the system calls implemented in this project. Each of these show the mapping between the specified system call and the values of the *arguments* member of the `system_call_arguments_t` structure. Consider the following function definition for the handler of spawn:

```
void spawn_syscall_handler(system_call_arguments_t* args);
```

The `args->call_id` would contain the value `SYS_SPAWN`. The name parameter for the `k_spawn` function would be contained in `args->arguments[4]` and the entry point function (func) is in `args->arguments[0]`. See the table under Spawn below.

Spawn

```
int Spawn(char *name, int (*func)(char *), char *arg, int stack_size, int priority, int *pid)
```

CALL_ID == SYS_SPAWN

Input Argument	<i>arguments</i> Position
name	arguments[4]
func	arguments[0]
arg	arguments[1]
stack_size	arguments[2]
priority	arguments[3]

Return Argument	<i>arguments</i> Position
pid	arguments[0]
Return value	arguments[3]

Wait

```
int Wait(int *pid, int *status)
```

CALL_ID == SYS_WAIT

Input Argument	<i>arguments</i> Position
None	

Return Argument	<i>arguments</i> Position
pid	arguments[0]
status	arguments[1]
Return value	arguments[3]

Exit

```
void Exit(int status)
```

CALL_ID == SYS_EXIT

Input Argument	<i>arguments</i> Position
status	arguments[0]

Return Argument	<i>arguments</i> Position
	None

SemCreate

```
int SemCreate(int value, int *semaphore)
```

CALL_ID == SYS_SEMCREATE

Input Argument	<i>arguments</i> Position
value	arguments[0]

Return Argument	<i>arguments</i> Position
Semaphore	arguments[0]
Return value	arguments[3]

SemP

```
int SemP(int semaphore)
```

CALL_ID == SYS_SEMP

Input Argument	<i>arguments</i> Position
semaphore	arguments[0]

Return Argument	<i>arguments</i> Position
Return value	arguments[3]

SemV

```
int SemV(int semaphore)
```

CALL_ID == SYS_SEMV

Input Argument	<i>arguments</i> Position
semaphore	arguments[0]

Return Argument	<i>arguments</i> Position
Return value	arguments[3]

SemFree

```
int SemFree(int semaphore)
```

CALL_ID == SYS_SEMFREE

Input Argument	<i>arguments</i> Position
semaphore	arguments[0]

Return Argument	<i>arguments</i> Position
Return value	arguments[3]

GetTimeOfDay

```
void GetTimeOfDay(int *tod)
```

CALL_ID == SYS_GETTIMEOFDAY

Input Argument	<i>arguments</i> Position
	None

Return Argument	<i>arguments</i> Position
tod	arguments[0]

CPUTime

```
void CPUTime(int *cpu)
```

CALL_ID == SYS_CPUTIME

Input Argument	<i>arguments</i> Position
	None

Return Argument	<i>arguments</i> Position
cpu	arguments[0]

GetPID

```
void GetPID(int *pid)
```

CALL_ID == SYS_GETPID

Input Argument	<i>arguments</i> Position
	None

Return Argument	<i>arguments</i> Position
pid	arguments[0]

5. Scheduler and Messaging Functions

For the System Calls project, you can make use of any of the Messaging project functions: **mailbox_create**, **mailbox_send**, **mailbox_receive**, **mailbox_release**, and **waitdevice**.

For Scheduler functions. You can make use of **k_spawn**, **k_wait**, **k_exit**, **k_join**, **k_kill**, **signaled**, **k_getpid**, **readtime**, and **displayProcessTable**. You CANNOT use **block**, **unblock**, **getStartTime**, or **timeSlice**.



Important: You cannot disable interrupts in the Messaging project. Instead, use mailboxes for mutual exclusion when necessary.